3.1 Java常见的集合类

面试官：说一说Java提供的常见集合？（画一下集合结构图）

候选人：

嗯~~，好的。

在java中提供了量大类的集合框架，主要分为两类：

第一个是Collection 属于单列集合，第二个是Map 属于双列集合

- 在Collection中有两个子接口List和Set。在我们平常开发的过程中用的比较多像list接口中的实现类ArrarList和LinkedList。 在Set接口中有实现类HashSet和TreeSet。
- 在map接口中有很多的实现类，平时比较常见的是HashMap、TreeMap，还有一个线程安全的map:ConcurrentHashMap

3.2 List

面试官：ArrayList底层是如何实现的？

候选人：

嗯~，我阅读过arraylist的源码，我主要说一下add方法吧

第一：确保数组已使用长度（size）加1之后足够存下下一个数据

第二：计算数组的容量，如果当前数组已使用长度+1后的大于当前的数组长度，则调用grow方法扩容（原来的1.5倍）

第三：确保新增的数据有地方存储之后，则将新元素添加到位于size的位置上。

第四：返回添加成功布尔值。

面试官：ArrayList list=new ArrayList(10)中的list扩容几次

候选人：

是new了一个ArrarList并且给了一个构造参数10，对吧？(问题一定要问清楚再答)

面试官：是的

候选人：

好的，在ArrayList的源码中提供了一个带参数的构造方法，这个参数就是指定的集合初始长度，所以给了一个10的参数，就是指定了集合的初始长度是10，这里面并没有扩容。

面试官：如何实现数组和List之间的转换

候选人：

嗯，这个在我们平时开发很常见

数组转list，可以使用jdk自动的一个工具类Arrars，里面有一个asList方法可以转换为数组

List 转数组，可以直接调用list中的toArray方法，需要给一个参数，指定数组的类型，需要指定数组的长度。

面试官：用Arrays.asList转List后，如果修改了数组内容，list受影响吗？List用toArray转数组后，如果修改了List内容，数组受影响吗

候选人：

Arrays.asList转换list之后，如果修改了数组的内容，list会受影响，因为它的底层使用的Arrays类中的一个内部类ArrayList来构造的集合，在这个集合的构造器中，把我们传入的这个集合进行了包装而已，最终指向的都是同一个内存地址

list用了toArray转数组后，如果修改了list内容，数组不会影响，当调用了toArray以后，在底层是它是进行了数组的拷贝，跟原来的元素就没啥关系了，所以即使list修改了以后，数组也不受影响

面试官：ArrayList 和 LinkedList 的区别是什么？

候选人：

嗯，它们两个主要是底层使用的数据结构不一样，ArrayList 是动态数组，LinkedList 是双向链表，这也导致了它们很多不同的特点。

1，从操作数据效率来说

ArrayList按照下标查询的时间复杂度O(1)【内存是连续的，根据寻址公式】， LinkedList不支持下标查询

查找（未知索引）： ArrayList需要遍历，链表也需要链表，时间复杂度都是O(n)

新增和删除

- ArrayList尾部插入和删除，时间复杂度是O(1)；其他部分增删需要挪动数组，时间复杂度是O(n)
- LinkedList头尾节点增删时间复杂度是O(1)，其他都需要遍历链表，时间复杂度是O(n)

2，从内存空间占用来说

ArrayList底层是数组，内存连续，节省内存

LinkedList 是双向链表需要存储数据，和两个指针，更占用内存

3，从线程安全来说，ArrayList和LinkedList都不是线程安全的

面试官：嗯，好的，刚才你说了ArrayList 和 LinkedList 不是线程安全的，你们在项目中是如何解决这个的线程安全问题的？

候选人：

嗯，是这样的，主要有两种解决方案：

第一：我们使用这个集合，优先在方法内使用，定义为局部变量，这样的话，就不会出现线程安全问题。

第二：如果非要在成员变量中使用的话，可以使用线程安全的集合来替代

ArrayList可以通过Collections 的 synchronizedList 方法将 ArrayList 转换成线程安全的容器后再使用。

LinkedList 换成ConcurrentLinkedQueue来使用

3.4 HashMap

面试官：说一下HashMap的实现原理？

候选人：

嗯。它主要分为了一下几个部分：

1，底层使用hash表数据结构，即数组+（链表 | 红黑树）

2，添加数据时，计算key的值确定元素在数组中的下标

key相同则替换

不同则存入链表或红黑树中

3，获取数据通过key的hash计算数组下标获取元素

面试官：HashMap的jdk1.7和jdk1.8有什么区别

候选人：

- JDK1.8之前采用的拉链法，数组+链表
- JDK1.8之后采用数组+链表+红黑树，链表长度大于8且数组长度大于64则会从链表转化为红黑树

面试官：好的，你能说下HashMap的put方法的具体流程吗？

候选人：

嗯好的。

1. 判断键值对数组table是否为空或为null，否则执行resize()进行扩容（初始化）
2. 根据键值key计算hash值得到数组索引
3. 判断table[i]==null，条件成立，直接新建节点添加
4. 如果table[i]==null ,不成立

4.1 判断table[i]的首个元素是否和key一样，如果相同直接覆盖value

4.2 判断table[i] 是否为treeNode，即table[i] 是否是红黑树，如果是红黑树，则直接在树中插入键值对

4.3 遍历table[i]，链表的尾部插入数据，然后判断链表长度是否大于8，大于8的话把链表转换为红黑树，在红黑树中执行插入操 作，遍历过程中若发现key已经存在直接覆盖value

5. 插入成功后，判断实际存在的键值对数量size是否超多了最大容量threshold（数组长度*0.75），如果超过，进行扩容。

面试官：好的，刚才你多次介绍了hsahmap的扩容，能讲一讲HashMap的扩容机制吗？

候选人：

好的

- 在添加元素或初始化的时候需要调用resize方法进行扩容，第一次添加数据初始化数组长度为16，以后每次每次扩容都是达到了扩容阈值（数组长度 * 0.75）
- 每次扩容的时候，都是扩容之前容量的2倍；
- 扩容之后，会新创建一个数组，需要把老数组中的数据挪动到新的数组中
- 没有hash冲突的节点，则直接使用 e.hash & (newCap - 1) 计算新数组的索引位置
- 如果是红黑树，走红黑树的添加
- 如果是链表，则需要遍历链表，可能需要拆分链表，判断(e.hash & oldCap)是否为0，该元素的位置要么停留在原始位置，要么移动到原始位置+增加的数组大小这个位置上

面试官：好的，刚才你说的通过hash计算后找到数组的下标，是如何找到的呢，你了解hashMap的寻址算法吗？

候选人：

这个哈希方法首先计算出key的hashCode值，然后通过这个hash值右移16位后的二进制进行按位**异或运算**得到最后的hash值。

在putValue的方法中，计算数组下标的时候使用hash值与数组长度取模得到存储数据下标的位置，hashmap为了性能更好，并没有直接采用取模的方式，而是使用了数组长度-1 得到一个值，用这个值按位与运算hash值，最终得到数组的位置。

面试官：为何HashMap的数组长度一定是2的次幂？

候选人：

嗯，好的。hashmap这么设计主要有两个原因：

第一：

计算索引时效率更高：如果是 2 的 n 次幂可以使用位与运算代替取模

第二：

扩容时重新计算索引效率更高：在进行扩容是会进行判断 hash值按位与运算旧数组长租是否 == 0

如果等于0，则把元素留在原来位置 ，否则新位置是等于旧位置的下标+旧数组长度

面试官：好的，我看你对hashmap了解的挺深入的，你知道hashmap在1.7情况下的多线程死循环问题吗？

候选人：

嗯，知道的。是这样

jdk7的的数据结构是：数组+链表

在数组进行扩容的时候，因为链表是**头插法**，在进行数据迁移的过程中，有可能导致死循环

比如说，现在有两个线程

线程一：**读取**到当前的hashmap数据，数据中一个链表，在准备扩容时，线程二介入

线程二也读取hashmap，直接进行扩容。因为是头插法，链表的顺序会进行颠倒过来。比如原来的顺序是AB，扩容后的顺序是BA，线程二执行结束。

当线程一再继续执行的时候就会出现死循环的问题。

线程一先将A移入新的链表，再将B插入到链头，由于另外一个线程的原因，B的next指向了A，所以B->A->B,形成循环。

当然，JDK 8 将扩容算法做了调整，不再将元素加入链表头（而是保持与扩容前一样的顺序），**尾插法**，就避免了jdk7中死循环的问题。

面试官：好的，hashmap是线程安全的吗？

候选人：不是线程安全的

面试官：那我们想要使用线程安全的map该怎么做呢？

候选人：我们可以采用ConcurrentHashMap进行使用，它是一个线程安全的HashMap

面试官：那你能聊一下ConcurrentHashMap的原理吗？

候选人：好的，请参考《多线程相关面试题》中的ConcurrentHashMap部分的讲解

面试官：HashSet与HashMap的区别？

候选人：嗯，是这样。

HashSet底层其实是用HashMap实现存储的, HashSet封装了一系列HashMap的方法. 依靠HashMap来存储元素值,(利用hashMap的key键进行存储), 而value值默认为Object对象. 所以HashSet也不允许出现重复值, 判断标准和HashMap判断标准相同, 两个元素的hashCode相等并且通过equals()方法返回true.

面试官：HashTable与HashMap的区别

候选人：

嗯，他们的主要区别是有几个吧

第一，数据结构不一样，hashtable是数组+链表，hashmap在1.8之后改为了数组+链表+红黑树

第二，hashtable存储数据的时候都不能为null，而hashmap是可以的

第三，hash算法不同，hashtable是用本地修饰的hashcode值，而hashmap经常了二次hash

第四，扩容方式不同，hashtable是当前容量翻倍+1，hashmap是当前容量翻倍

第五，hashtable是线程安全的，操作数据的时候加了锁synchronized，hashmap不是线程安全的，效率更高一些

> 在实际开中不建议使用HashTable，在多线程环境下可以使用ConcurrentHashMap类